

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automation

Zavtoni Ion

FAF-252

Report

Lab 1: Logging library

Object Oriented Programming

Programming language: Java

Checked by:
Dominic Flocea

Chişinău – 2026

1. Program Structure

1.1. Purpose and scope

This laboratory combines two small tasks: a logging library and a checker for round parentheses. The checker returns whether a string is valid, while the logger explains the steps taken to reach that answer. The program uses Java and its standard library; it does not require a separate logging framework.

The implementation supports the required levels, configurable message formatting, shared logger settings, and the optional random-phrase placeholder. Its four demonstration inputs are the ones listed in the handout. The project is opened in IntelliJ IDEA and started from `Main.main()`.

1.2. Responsibilities of the four types

Source file	Responsibility
`Main.java`	Configures logging, creates a validator, and passes each example string to it.
`LogLevel.java`	Defines the four levels, their severity values, and their possible random phrases.
`Logger.java`	Stores the logger's name, filters messages, substitutes placeholders, and prints accepted entries.
`ParenthesesValidator.java`	Checks the order and matching of parentheses and reports events through a logger.

1.3. How the parts collaborate

`Main` creates one `ParenthesesValidator`. That validator contains a reference to a `Logger` named `Parentheses`. The logger consults `LogLevel` when deciding whether a message should appear and when choosing a random phrase.

Main → ParenthesesValidator → Logger → LogLevel

This is a *uses* relationship. The validator does not extend `Logger`: checking parentheses and printing diagnostic messages are different responsibilities. The logger can therefore also be used by another program.

The project repository is itsjohnyoff/OOP-course. This report explains the classes, their collaboration, and the checks performed on the current project. The following sections connect each laboratory requirement to the corresponding implementation behavior.

2. Classes and Objects

2.1. Types, instances, and references

A class defines the fields and methods available to its objects. The validator class provides `isValid()`. The `Logger` class provides methods such as `info()` and `error()`. The following statement in `Main` creates an actual validator object:

```
ParenthesesValidator validator = new ParenthesesValidator();
```

The first `ParenthesesValidator` is the variable's type. The name `validator` identifies a local variable. The expression using `new` creates an object and invokes its constructor. The variable stores a reference to that object, which is used for calls such as `validator.isValid("()")`. No explicit constructor is declared in the validator, so Java supplies a default no-argument constructor.

2.2. What state each object stores

The validator has one instance field:

```
private final Logger logger = new Logger("Parentheses");
```

Here, `Logger` is a type and `logger` is a field name. The string `"Parentheses"` is the name stored by the created logger, not the Java variable name. Another validator object would initialize its own logger object.

A logger stores an instance field called `name`. In contrast, `level` and `format` are shared class fields. The validator's input, character index, current character, and balance are method parameters or local variables. They are not stored as persistent validator fields. Every call therefore starts a fresh validation count.

2.3. Encapsulation and the final reference

`private` prevents code outside the validator from directly accessing its `logger` field. Other classes ask the validator to perform its job through the public `isValid()` method. They do not need access to its diagnostic collaborator.

`final` prevents reassignment of that field after initialization. It does not prevent calls to `logger.info(...)` and does not make all logger configuration immutable. Likewise, the logger's `private final String name` is assigned during construction and cannot later be replaced through that field.

`Main.main()` is static, so the entry point is invoked without creating a `Main` object. `LogLevel` is an enum: its constants are predefined instances carrying severity and phrase data.

3. Static and Instance Members

3.1. One name per object; one shared configuration

The distinction is visible directly in `Logger.java`:

```
private final String name;
private static LogLevel level = LogLevel.INFO;
private static String format = "[%t] [%l] %n: %m";
```

The `name` field belongs to each logger object. Two objects can therefore identify different parts of a program. The `static` fields belong to the class and are consulted by every logger instance. Storing the minimum level and format this way gives all loggers one common configuration.

The setters are also static, so they are called through the class:

```
Logger.setLevel(LogLevel.TRACE);
Logger.setFormat("[%t] [%l] %n: %m | %r");
```

These are the settings chosen by `Main`. They override the initial defaults before the four examples are checked. The setter methods reject null values using `Objects.requireNonNull()`.

3.2. How the minimum level works

The enum assigns severity values 10, 20, 30, and 40 to TRACE, INFO, WARN, and ERROR. Its comparison is:

```
public boolean allows(LogLevel messageLevel) {
    return messageLevel.severity >= this.severity;
}
```

When called as `level.allows(messageLevel)`, `this` is the configured minimum level. A message is accepted when its severity is at least that minimum.

Configured level	Messages printed
TRACE	TRACE, INFO, WARN, ERROR
INFO	INFO, WARN, ERROR
WARN	WARN, ERROR
ERROR	ERROR only

The static `TIME_FORMAT` field is different from the mutable configuration: it is a shared final reference to the reusable `DateTimeFormatter` for `HH:mm:ss` timestamps. The enum's `severity` and `phrases` are instance fields belonging to each enum constant.

3.3. Demonstrating shared settings

The following small experiment uses two separate logger objects. It is a verification example rather than an additional feature of `Main`.

```
Logger first = new Logger("First");
Logger second = new Logger("Second");

Logger.setLevel(LogLevel.ERROR);
Logger.setFormat("%n/%l/%m");
first.info("hidden");
first.error("one");
second.error("two");
```

Expected output:

```
First/ERROR/one
Second/ERROR/two
```

Both objects use the new level and format, even though they were created before the setters were called. Their names remain different because the name belongs to each object. The INFO entry is filtered out before formatting and printing.

Changing the shared configuration again affects existing objects:

```
Logger.setLevel(LogLevel.TRACE);
Logger.setFormat("[%l] %n: %m");
first.trace("visible now");
second.info("same configuration");
```

This time, both messages are accepted. Creating a third logger would not create a third independent level or format: it would consult the same static fields.

3.4. Why these choices fit the laboratory

Instance names identify the source of an entry, while static configuration avoids setting the same options separately on every object. Keeping the fields private means configuration changes go through the public setters, where invalid null values can be rejected.

The shared settings also have a consequence: changing the format for one demonstration changes it for every subsequent logger call. Tests must restore the configuration they depend on. The current application uses one thread and changes settings before validation; it does not claim to provide synchronized configuration for concurrent callers.

The validator's final logger reference and the logger's static settings solve different problems. The final reference prevents replacement of a collaborator. Static fields establish shared state; they do not prevent that state's values from changing.

4. Methods and Responsibilities

4.1. The logger's message pipeline

The public methods `trace()`, `info()`, `warn()`, and `error()` are convenient entry points. Each supplies its level to `log()`. For example:

```
public void info(String message) {
    log(LogLevel.INFO, message);
}
```

The responsibility of `log()` is to reject null arguments, apply the minimum-level filter, and print an accepted message. Formatting is delegated to the private `formatMessage()` helper.

```
if (!level.allows(messageLevel)) {
    return;
}
System.out.println(formatMessage(messageLevel, message));
```

This early `return` leaves the void logging method without printing. It does not stop validation, because the validation method continues after its logger call.

4.2. Formatting an accepted entry

The formatter scans a local copy of the template and appends output to a `StringBuilder`. Ordinary characters are copied. A recognized percent placeholder is replaced, and its letter is skipped so that it is not printed twice.

Placeholder	Replacement
`%t`	Current local time formatted as `HH:mm:ss`.
`%l`	Level of this entry, such as INFO.
`%n`	This logger object's name.
`%m`	Original message text.
`%r`	Random phrase associated with this entry's level.

An unknown placeholder such as `%x` is preserved. A final percent character is preserved too. Values inserted from a name or message are not scanned again, so a message containing `%l` keeps that literal text.

For `%r`, `messageLevel.randomPhrase()` selects from that enum constant's phrases using `ThreadLocalRandom`. The formatter caches the choice within one entry, so repeated `%r` placeholders agree. A later entry may choose another phrase. The choice affects presentation, never the validation answer.

4.3. The validator's character loop

`isValid(String input)` receives one complete string and returns a primitive `boolean`. Null is rejected before accessing the string's length. The method then initializes a local balance to zero and reads one character at a time:

```
int balance = 0;
for (int i = 0; i < input.length(); i++) {
    char current = input.charAt(i);
    // One branch handles the current character.
}
```

The branches are mutually exclusive. An opening increases balance and produces TRACE. A closing first checks whether an unmatched opening is available. If balance is zero, the validator warns and returns an invalid result immediately. Otherwise it decreases balance and logs a completed pair at INFO. Any other character is rejected.

For `(())`, the state changes as follows:

Index	Character	Balance	Event
0	(	1	TRACE: opening found
1	(	2	TRACE: opening found
2	)	1	INFO: matching pair
3	)	0	INFO: matching pair

After the loop, a positive balance means some openings remain unmatched. A zero balance means the input is valid, provided no earlier rejection occurred. Indexes in messages are zero-based.

4.4. Reporting failure without duplicating code

```
private boolean reportInvalid(String input, String reason) {
    logger.error("Invalid input \"" + input + "\": " + reason);
    logger.info("Validation result for \"" + input + "\": false");
    return false;
}
```

`return reportInvalid(input, reason)` first executes this helper, then returns its false result from `isValid()`. It ends only that call, so `Main`'s loop can check the next string. This is a method call between collaborating objects when the helper invokes `logger.error(...)`. The separate logger keeps filtering and formatting out of the validation algorithm.

5. Testing and Correctness

5.1. The four required examples

The demonstration in `Main` uses the exact input strings required by the handout. TRACE is enabled and the format is `[%t] [%l] %n: %m | %r`. Therefore timestamps, opening events, completed pairs, failures, and final results are visible.

Input	Expected	Observed	Reason
`()`	true	true	One opening followed by its closing.
`)(((()))`	false	false	An unmatched closing appears at index 0.
`(`	false	false	One unmatched opening remains at the end.
`(())((()())())`	true	true	All pairs match in the correct order.

The required inputs passed. The final Boolean returned by `isValid()` agrees with the final INFO message. `Main` currently ignores that returned value, because the validator already reports its result through the logger.

5.2. Reading one successful run

For `()`, the core messages are:

```
[TRACE] Parentheses: Opening parenthesis found at index 0; unmatched openings: 1
[INFO] Parentheses: Matching pair completed at index 1
[INFO] Parentheses: Validation result for "()": true
```

Timestamps and the random suffix have been omitted here for readability. The first line follows `balance++`; the second follows `balance--`; the third follows the successful end-of-input check. The current format includes `%t`, so the actual console output displays the local time before each level.

For `)(((()))`, checking stops at the first character. It is unnecessary to inspect the remaining characters: a later opening cannot repair a closing that has already appeared too early. The WARN line describes the unmatched closing, and `reportInvalid()` supplies the ERROR explanation and false final result.

An ERROR line in this output is a deliberate diagnostic entry, not an uncaught Java exception. Invalid example inputs are expected test cases, and the loop proceeds to the next example.

5.3. Additional validation cases

The required examples cover basic success and failure, but extra cases check boundaries and repeated use of one validator object.

Input	Result	Behavior checked
Empty string	true	No characters and no unmatched openings.
`()()`	true	Consecutive independent pairs.
`(((())))`	true	Several nested pairs.
`)(`	false	Correct total counts do not imply correct order.
`())`	false	Extra closing after a completed pair.
`(()`	false	An opening remains unmatched.
`(a)`	false	Letters are rejected.
`( )`	false	Spaces are rejected rather than ignored.
Null reference	false	Rejected before calling `length()`.

The `)(` case is especially useful for preventing an incorrect solution that compares only the total number of openings and closings. Its totals are equal, but its first closing has no available opening. The early `balance == 0` check is necessary.

5.4. A fresh count for each call

The same object was checked with an invalid input followed by a valid one. The valid input still returned true. This checks that the previous call's state does not leak into the next call. The declaration `int balance = 0` inside `isValid()` ensures a new count for every invocation.

The verification also included 2,000 nested pairs and a string of 2,000 unmatched openings. These cases exercise a longer scan without recursion or a stack in the submitted validator. They do not establish a general performance benchmark; they check the behavior of the same counter logic on longer inputs.

Null input is handled as a validation failure, while null arguments supplied directly to the logger are rejected by its argument checks. These are separate contracts: `isValid(null)` returns false, but constructing `new Logger(null)` is not accepted.

5.5. Checking filtering and shared configuration

A temporary standard-library test harness was compiled against the current four source files. It is kept outside the submitted `src` tree. The verification completed with:

```
All 2512 checks passed.
```

The checks include assertions within repeated and generated cases; the number does not mean 2,512 separate hand-written test methods. Every parentheses string of length 0 through 10 was compared with an independent stack-based reference checker. That contributes 2,047 generated inputs.

Filtering was checked at all four configured levels by sending one entry of each level and comparing the visible output with the table in Section 3. Validation results were also checked at every level. Raising the minimum hides some entries, but it does not change true to false or false to true.

5.6. Formatting and bonus checks

The formatting checks covered `%t`, `%l`, `%n`, and `%m`, as well as repeated placeholders, an empty template, unknown placeholders, and a trailing percent sign. The time value was checked for the `HH:mm:ss` shape and against the current time around the logging call.

The literal-substitution case used placeholder-like text inside a logger name and message. That text remained unchanged after insertion. This checks that the formatter processes only its template, rather than repeatedly replacing tokens inside the output it has already built.

For the bonus, multiple messages were logged at each level. Each generated phrase was checked against the phrases for that entry's level. Two `%r` placeholders in one template were also checked for equality within that entry. The checks do not require different runs to choose different phrases: random selection may legitimately repeat a value.

Shared-state checks used separate logger objects with different names. Updating the static settings affected both existing objects and an object created later, while all three retained their individual names. The harness restored the standard level and format after completion so that test configuration would not be mistaken for the application defaults.

5.7. Why the counter algorithm is correct

Before each character is processed, balance represents the number of openings in the already-read prefix that have not yet been matched. Initially the prefix is empty and balance is zero.

An opening adds one unmatched opening, so incrementing balance preserves that meaning. A closing consumes one available opening, so decrementing a positive balance preserves it too. If balance is zero when a closing appears, the prefix is already invalid; no later character can change the order of that prefix. Rejecting immediately is therefore correct.

If the entire string is processed without rejection, a positive final balance means openings remain. A zero final balance means every opening has a closing and every closing had an earlier opening. Both matching and ordering requirements are satisfied. Characters outside the permitted alphabet are rejected explicitly.

5.8. Cost and practical limits

For an input of length `n`, the validation scan visits each character at most once. Its decision logic takes $O(n)$ time in the worst case and uses $O(1)$ auxiliary space: a balance counter, an index, and the current character. An early failure can inspect fewer than `n` characters.

These bounds describe the validation algorithm. Logging also constructs strings, scans a format template, and writes output. In particular, a final message contains the input itself, so diagnostic text can require space proportional to the input length. The overall program should not be described as allocating only constant space.

5.9. What the results demonstrate

The required examples and additional checks support the implemented behavior for round parentheses. They also check that presentation choices do not alter the Boolean decision. They are not evidence that the logger is a production logging framework.

The implementation intentionally uses the console, the system clock, and shared mutable configuration. It has no log-file output, asynchronous processing, or defined concurrent-update behavior. The input policy is strict: spaces, letters, square brackets, and curly brackets are invalid. Extending those policies would require an explicit design change rather than a different logging format.

6. Reflection

6.1. The most important OOP concept

The main design lesson is the separation of responsibilities through collaborating objects. The validator owns the correctness decision. Its logger owns the presentation of diagnostic events. Calling `logger.info(...)` lets the validator describe an event without implementing severity comparisons, timestamps, or template parsing itself.

Encapsulation makes this relationship clear. Outside code calls `isValid()` instead of reaching into the private logger field. The final reference expresses that the validator keeps the same collaborator after construction. At the same time, the static configuration demonstrates shared state: logger objects have separate names but consult one common level and format.

This implementation also makes the distinction between object state and method-local state concrete. The logger reference survives between calls. The balance counter does not; it starts at zero each time. Moving balance into an instance field without resetting it would create a risk of incorrect results across calls.

6.2. Improvements to consider

One useful improvement would be an optional constructor that accepts a logger. A default constructor could still create the named console logger for the laboratory. Supplying the collaborator would make it easier to capture output or replace diagnostics during testing without changing the parentheses algorithm.

Another improvement would be to make the output destination and time source configurable. Direct calls to `System.out.println()` and `LocalTime.now()` are sufficient for this demonstration, but an injected output stream and clock would make logger checks more deterministic. The required shared level and format could remain static.

For a multi-threaded application, configuration updates would need a defined synchronization strategy. Simply copying the format reference into a local variable does not make all shared settings thread-safe. That additional machinery is outside the scope of the current single-threaded program.

6.3. Explaining the implementation

The central questions are answerable directly from the source: why a name belongs to an object, why level and format are static, why `)(` fails despite equal character counts, and why a filtered INFO message does not change the returned Boolean. Understanding those points is more useful than memorizing console output, especially because the random suffix can change between runs.

6.4. Requirement coverage

Handout requirement	Implementation evidence
Named logger object	`Logger.name`, assigned by its constructor.
Shared configurable settings	Static `level` and `format`, updated by setters.
Four levels and minimum filtering	Enum severities and `allows()`, called from `log()`.
Time, level, name, and message placeholders	Corresponding cases in `formatMessage()`.
Boolean parentheses validation	Local balance and rejection paths in `isValid()`.
TRACE on openings	`logger.trace(...)` after incrementing balance.
INFO on pairs and final result	Matching-pair call and final-result calls.
Invalid string and explanation	`reportInvalid(input, reason)` logs ERROR.
Optional warning	WARN before rejecting an unmatched closing.
Extra-credit random phrases	Enum phrase lists and the formatter's `r` case.
Required four input examples	The array and loop in `Main.main()`.

6.5. Sources and reproducibility

1. Dominic Flocea, *Lab 1: Logging library*, October 2026, supplied handout `h01-logging.pdf`. Requirements and the six report sections follow this handout.
2. Project source: github.com/itsjohnyoff/OOP-course, directory `lab1/src`. This report describes the current Java implementation, including the timestamp format added on 30 October 2026.

The code was compiled and checked with JDK 21. The console demonstration is reproduced by running `Main.main()` in IntelliJ IDEA. Compilation produces an `out` directory; it is generated output rather than additional source code.

The report consists of this PDF and its editable `main.tex` and `content.tex` sources, with the cover logo in `images/`. The report does not change application behavior. Required examples have fixed Boolean answers, while timestamps and random phrases are expected to vary.